ALARD UNIVERSITY PUNE
ATTITUDE • LEADERSHIP • ALERTNESS • READINESS • DEDICATION

OS MINI PROJECT REPORT

SMART DISK SCHEDULING AND PERFORMANCE ANALYZER

Submitted by:

Adiba Khan

Roll No.: 03

PRN: 25ET2041003

Guided by:

Prof.Krati Shrivastava

Alard School of Engineering and Technology
Alard University Pune

ABSTRACT:

Disk scheduling is an important function of an Operating System that determines the order in which disk input/output requests are processed. An appropriate scheduling strategy can reduce disk head movement and improve the efficiency of disk access.

The Smart Disk Scheduling and Performance Analyzer Using C is a mini-project developed to implement and compare three disk scheduling algorithms: First Come, First Served (FCFS), Shortest Seek Time First (SSTF), and SCAN.

The program accepts disk size, request queue, and initial disk head position as input. It calculates the total head movement for each algorithm and displays the results for comparison.

This project helps students understand disk scheduling techniques and analyze how different algorithms behave for the same input.

Introduction:

Disk scheduling is an Operating System function that decides the order of pending disk I/O requests.

The disk head moves between cylinder positions to access requested data.

Different algorithms produce different amounts of total head movement.

This project implements and compares FCFS, SSTF, and SCAN using C.

Problem Statement :

Multiple disk requests may be waiting at the same time.
The operating system needs a strategy to choose the order in which requests are serviced.
Different strategies can cause different amounts of disk head movement.
The project calculates and compares head movement for three scheduling algorithms.

Objectives:

- Study disk scheduling concepts in Operating Systems.
- Implement FCFS, SSTF, and SCAN algorithms using C.
- Accept disk size, request queue, and initial head position.
- Calculate total head movement for each algorithm.
- Compare results and report the least movement for the given input.

Software and Hardware Requirements:

- Programming language: C
- Compiler: GCC or another standard C compiler
- Operating system: Windows or Linux
- Editor: VS Code or any text editor
- Hardware: Computer or laptop with a C compiler installed

Theory:

1.FCFS — First Come, First Served:-

Processes requests in the same order in which they are provided.
Advantages: simple to implement and follows the given request order.
Limitation: may cause large total head movement.
Best understood as a straightforward baseline for comparison.

2.SSTF — Shortest Seek Time First:-

Selects the pending request closest to the current disk head position.
Often reduces total head movement compared with FCFS.
The closest request is selected repeatedly until all requests are processed.
Limitation: requests far from the current position may wait longer.

3. SCAN — Elevator Algorithm:-

Moves the disk head in one direction and services requests along the way.
Reaches the end of the disk, then reverses direction.
This implementation initially moves toward higher-numbered cylinders.
Total movement depends on the initial head position and request queue.

System Design:

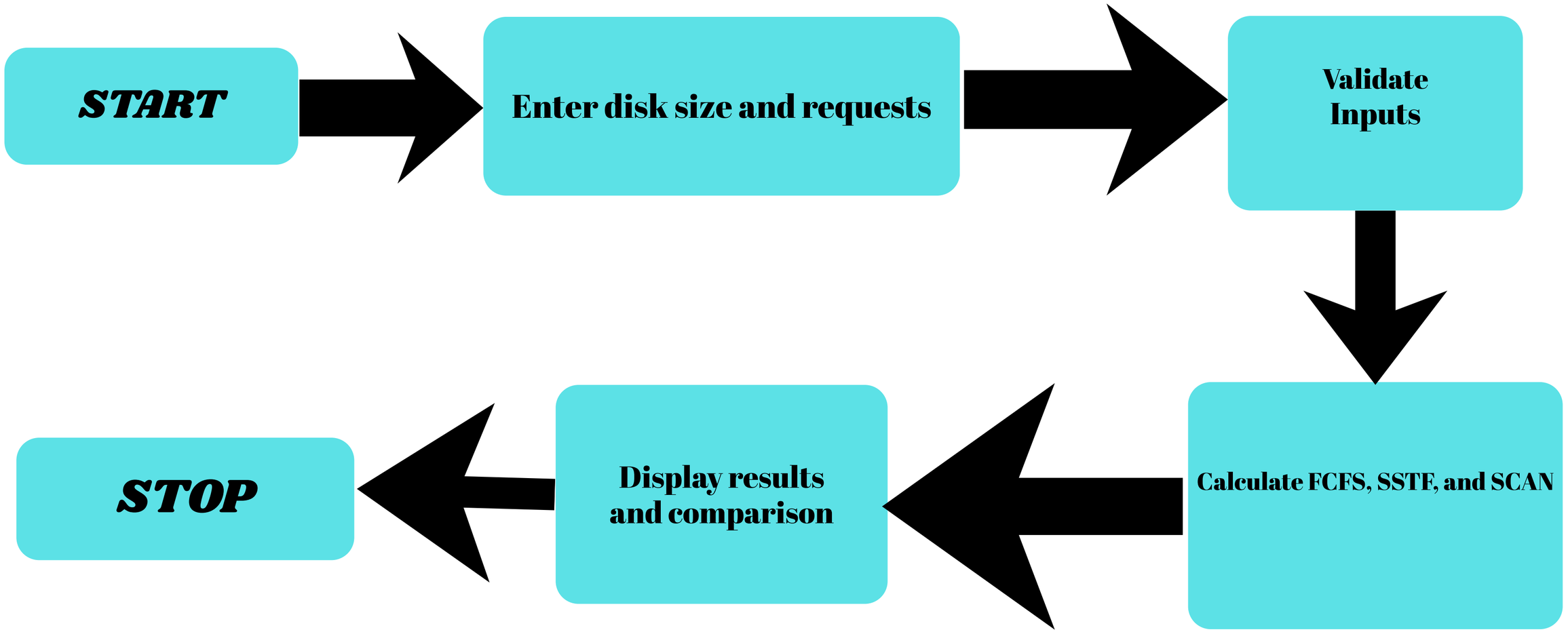
INPUT: disk size, number of requests, request queue, and initial head position.

PROCESS: run FCFS, SSTF, and SCAN independently on the same requests.

CALCULATION: add absolute differences between consecutive head positions.

OUTPUT: show total head movement and identify the lowest value.

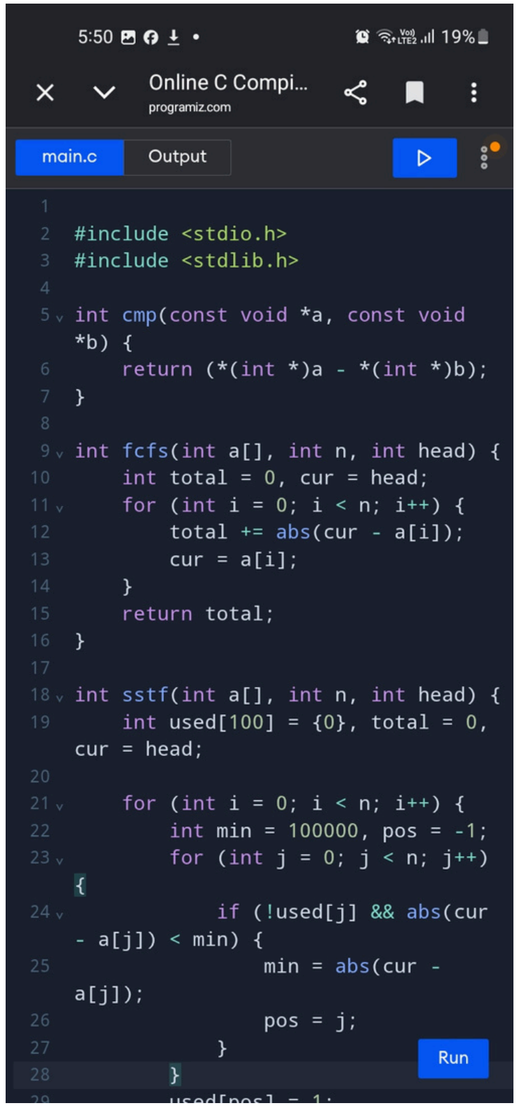
Program Flow:



Algorithm Steps:

1. Start the program.
2. Input the disk size.
3. Input the number of disk requests.
4. Enter the disk request queue.
5. Enter the initial disk head position.
6. Validate the input values.
7. Calculate total head movement using FCFS.
8. Calculate total head movement using SSTF.
9. Calculate total head movement using SCAN.
10. Display the results of all three algorithms.
11. Compare the results and display the least head movement.
12. Stop the program.

SOURCE CODE:



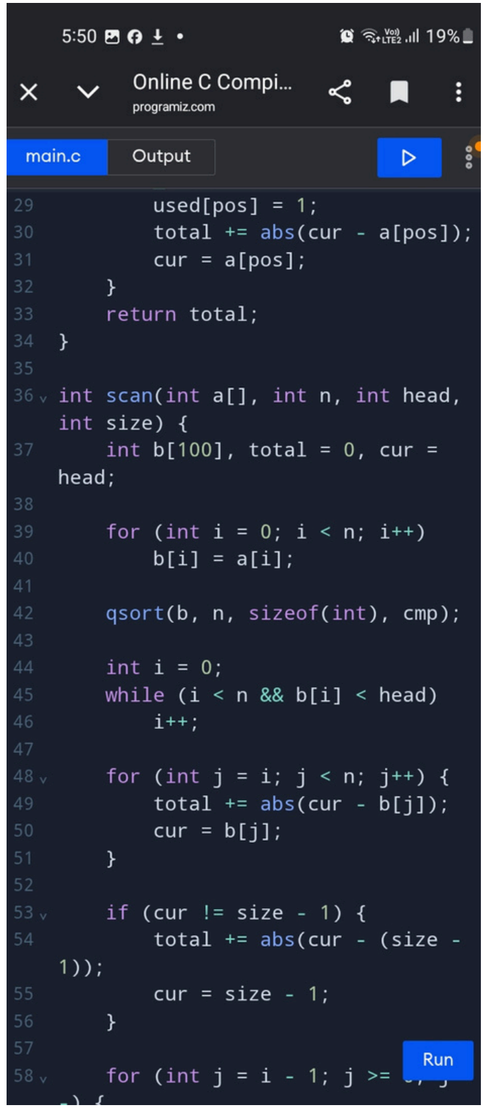

```c
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int fcfs(int a[], int n, int head) {
    int total = 0, cur = head;
    for (int i = 0; i < n; i++) {
        total += abs(cur - a[i]);
        cur = a[i];
    }
    return total;
}

int sstf(int a[], int n, int head) {
    int used[100] = {0}, total = 0, cur = head;

    for (int i = 0; i < n; i++) {
        int min = 100000, pos = -1;
        for (int j = 0; j < n; j++) {
            if (!used[j] && abs(cur - a[j]) < min) {
                min = abs(cur - a[j]);
                pos = j;
            }
        }
```




```c
        used[pos] = 1;
        total += abs(cur - a[pos]);
        cur = a[pos];
    }
    return total;
}

int scan(int a[], int n, int head, int size) {
    int b[100], total = 0, cur = head;

    for (int i = 0; i < n; i++)
        b[i] = a[i];

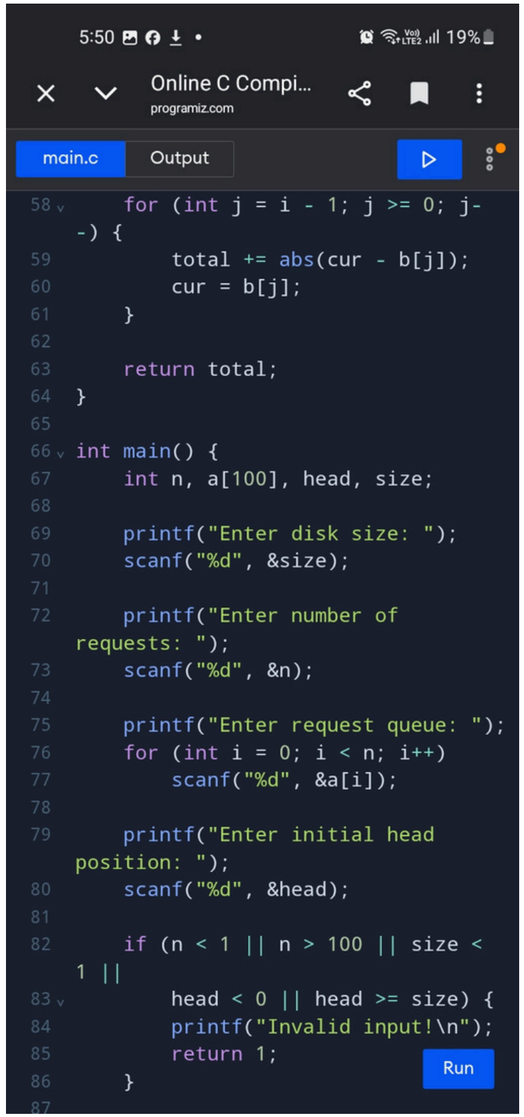
    qsort(b, n, sizeof(int), cmp);

    int i = 0;
    while (i < n && b[i] < head)
        i++;

    for (int j = i; j < n; j++) {
        total += abs(cur - b[j]);
        cur = b[j];
    }

    if (cur != size - 1) {
        total += abs(cur - (size - 1));
        cur = size - 1;
    }

    for (int j = i - 1; j >=
```




```c
    for (int j = i - 1; j >= 0; j--) {
        total += abs(cur - b[j]);
        cur = b[j];
    }

    return total;
}

int main() {
    int n, a[100], head, size;

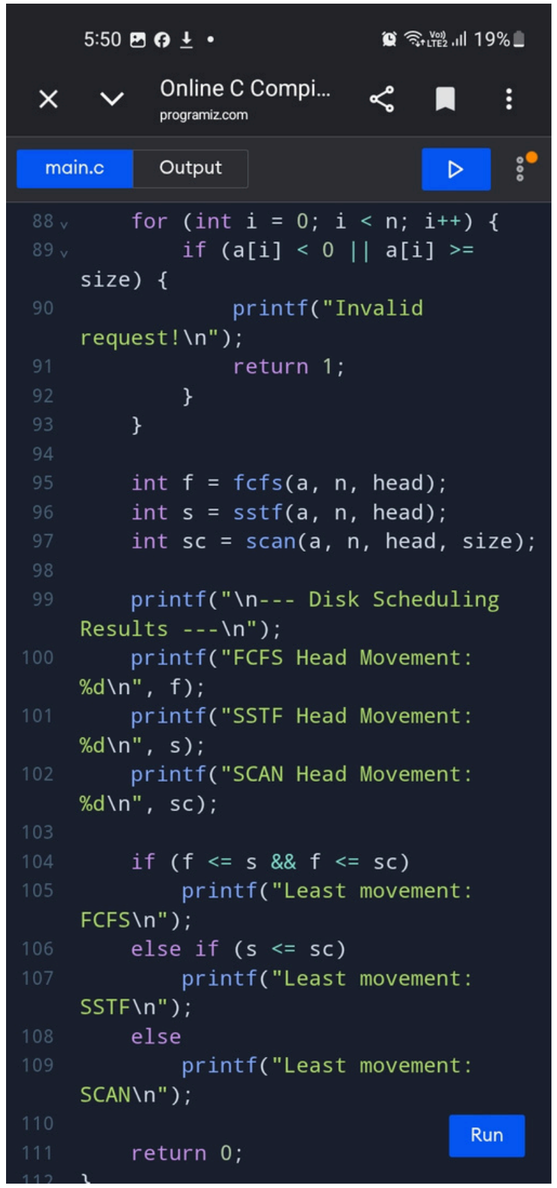
    printf("Enter disk size: ");
    scanf("%d", &size);

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    if (n < 1 || n > 100 || size < 1 ||
        head < 0 || head >= size) {
        printf("Invalid input!\n");
        return 1;
    }
```



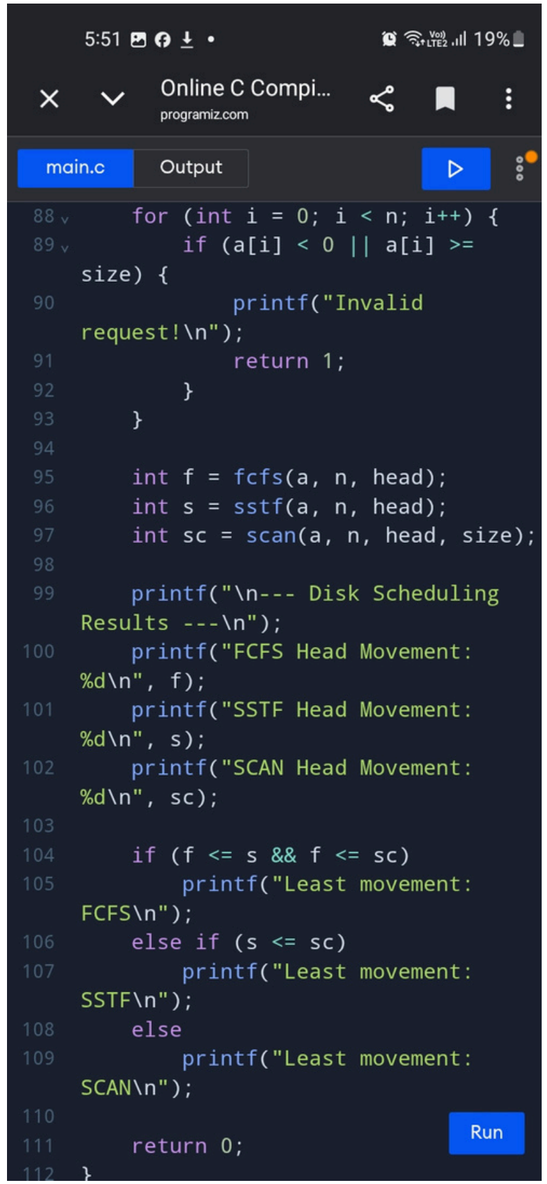

```c
    for (int i = 0; i < n; i++) {
        if (a[i] < 0 || a[i] >= size) {
            printf("Invalid request!\n");
            return 1;
        }
    }

    int f = fcfs(a, n, head);
    int s = sstf(a, n, head);
    int sc = scan(a, n, head, size);

    printf("\n--- Disk Scheduling Results ---\n");
    printf("FCFS Head Movement: %d\n", f);
    printf("SSTF Head Movement: %d\n", s);
    printf("SCAN Head Movement: %d\n", sc);

    if (f <= s && f <= sc)
        printf("Least movement: FCFS\n");
    else if (s <= sc)
        printf("Least movement: SSTF\n");
    else
        printf("Least movement: SCAN\n");

    return 0;
}
```




```c
    for (int i = 0; i < n; i++) {
        if (a[i] < 0 || a[i] >= size) {
            printf("Invalid request!\n");
            return 1;
        }
    }

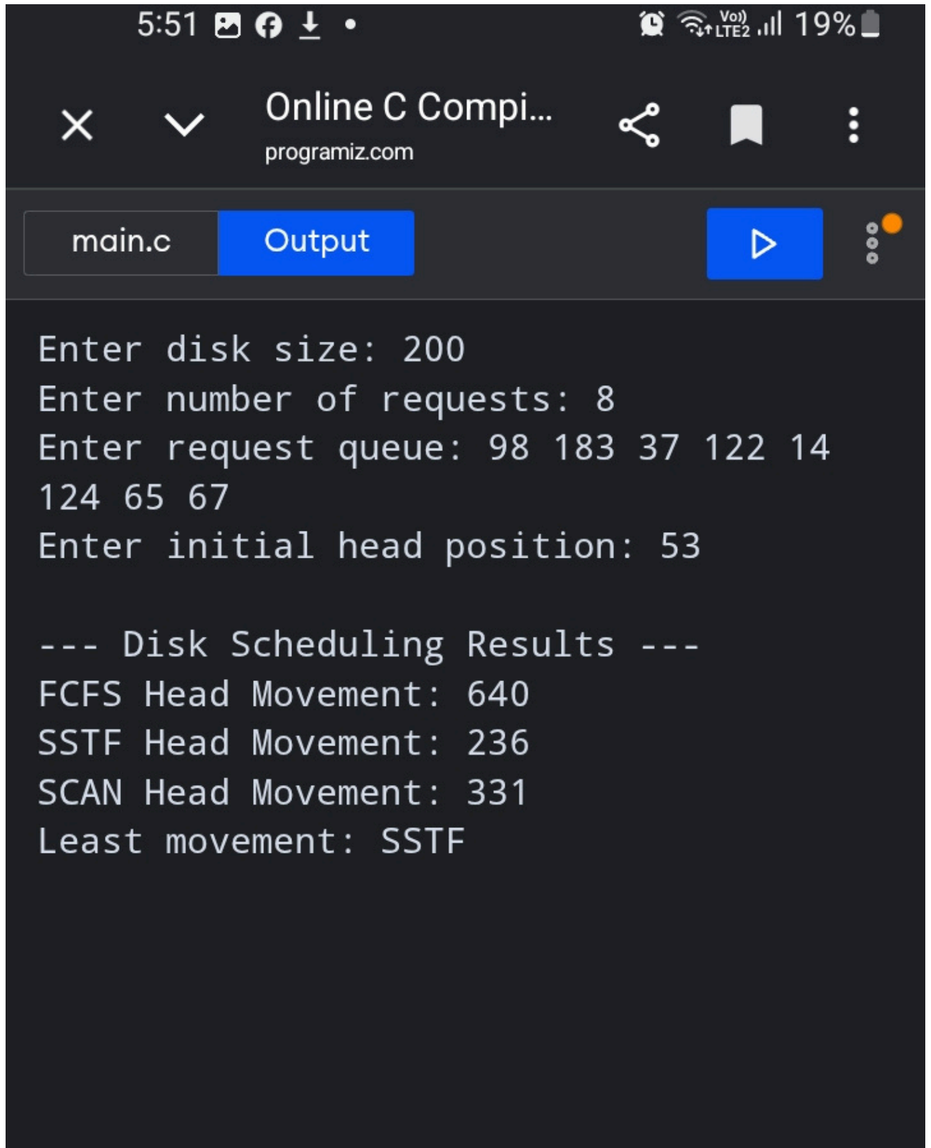
    int f = fcfs(a, n, head);
    int s = sstf(a, n, head);
    int sc = scan(a, n, head, size);

    printf("\n--- Disk Scheduling Results ---\n");
    printf("FCFS Head Movement: %d\n", f);
    printf("SSTF Head Movement: %d\n", s);
    printf("SCAN Head Movement: %d\n", sc);

    if (f <= s && f <= sc)
        printf("Least movement: FCFS\n");
    else if (s <= sc)
        printf("Least movement: SSTF\n");
    else
        printf("Least movement: SCAN\n");

    return 0;
}
```

Sample Input/ Output:



RESULT:

Algorithm	Total Head Movement
FCFS	640 cylinders
SSTF	236 cylinders
SCAN	331 cylinders

- For this input, SSTF produces the least head movement.
- The result can change when the request queue or initial head position changes.
- Head movement is a comparison metric; it is not a direct measurement of real disk access time.

Applications and Limitations:

- Applications: Operating System learning, laboratory demonstrations, and algorithm comparison.
- The program uses simulated requests; it does not control a physical disk.
- It calculates head movement, not actual disk access time.
- SSTF may cause long waiting times for some requests.
- Only FCFS, SSTF, and SCAN are included in this version.

Future Scope:

- Add C-SCAN and LOOK algorithms.
- Display the order in which requests are serviced.
- Generate graphs to compare the algorithms.
- Calculate additional metrics such as average movement per request.
- Develop a graphical user interface.

Conclusion:

- The project demonstrates FCFS, SSTF, and SCAN disk scheduling algorithms using C.
- It calculates and compares total disk head movement for the same request queue.
- For the sample input, SSTF produces the lowest movement.
- The project builds practical understanding of disk scheduling in Operating Systems.

References:

- Silberschatz, Galvin, and Gagne — Operating System Concepts.
- William Stallings — Operating Systems: Internals and Design Principles.
- Operating System lecture notes and laboratory material.
- C programming language reference material.